NoSQL Injections

When client input changes a database query

A controlled MongoDB investigation

CS440 Group Project

Documents and query predicates

SYNTHETIC ACCOUNT

```
{ username: "alice",
  role: "student" }
```

LITERAL EQUALITY

```
{ username: "alice" }
```

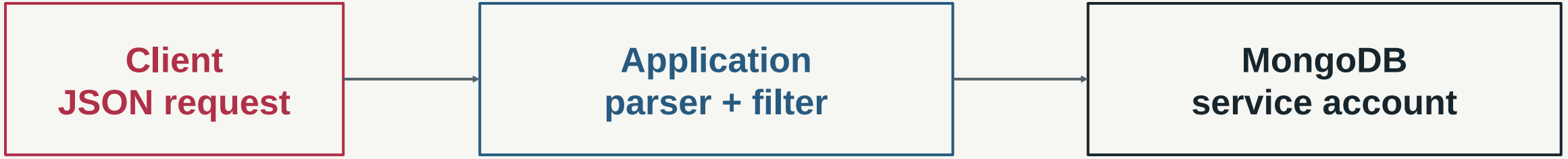
OPERATOR PREDICATE

```
{ username: { $ne: "" } }
```

NoSQL families: document, key-value, graph and wide-column
Flexible records and distributed scaling motivate adoption

MongoDB $ne documentation; Ron et al., 2016

Application architecture and threat model



Trust boundary: the app accepts a value as a database predicate

Attacker: JSON body and known username, no direct DB access
App: matching account creates a session
Consequence: false identity exposes protected data

The assigned paper and our investigation

Attack surface	Our treatment
Query conditions and operators	Technical deep dive
JavaScript execution	Separate control comparison
Command boundaries	Historical context
Browser-origin violations	Adjacent trust boundary

Research question: which controls preserve this login query’s intent?

Ron, Shulman-Peleg & Puzanov. IEEE Security & Privacy 14(2), 30-39 (2016)

An object changes the password condition

CLIENT REQUEST

```
{"username":"alice","password":{"$ne":""}}
```

VULNERABLE APPLICATION CODE

```
users.findOne({
  username: body.username,
  password: body.password
})
```

Alice matches when her stored password is non-empty

A matched record incorrectly becomes an authenticated session

C04 reproduction; MongoDB $ne documentation

Input shape and attack assumptions

OPERATOR OBJECT

```
"password": { "$ne": "" }
```

LITERAL STRING

```
"password": "{\"$ne\":\"\"}"
```

Known unique username. Non-empty stored value. Session on match.

Lab simplification: plaintext toy query; hardened path uses hash-only records

C04 object attack and C06 literal-string control; synthetic plaintext reproduction

Controlled experiment

15 request cases × 5 configurations × 2 runs

150 configuration/case observations

Login status and session decision
Protected-page access
Actual database find commands

Live demonstration

Step	Request and configuration	Observed outcome
1	Wrong password / baseline	401; access denied
2	Operator object / baseline	200; Alice session
3	Same object / hardened	400; zero find commands
4	Valid credentials / hardened	200; Alice session

Open local lab: http://127.0.0.1:4400

Results identify the effective boundary

Configuration	C04 login	Find commands	Valid login
Baseline	200 / Alice	1	200
JavaScript disabled	200 / Alice	1	200
Read-only account	200 / Alice	1	200
String validation only	400 / rejected	0	200
Full hardened path	400 / rejected	0	200

Read permission and scripting controls do not enforce this credential contract

Measured C01, C02 and C04; 15 cases × 5 configurations × 2 repetitions

Validation and independent credential verification

ISOLATED FIX

```
typeof username === "string"
typeof password === "string"
```

COMPLETE AUTHENTICATION PATH

Validate request contract
Look up account by username
Verify password hash before creating a session

src/server.mjs, src/passwords.mjs; scrypt N=131072, r=8, p=1

Historical examples and current scope

2016

Assigned paper: attack mechanisms and lifecycle mitigations

TESTED ENVIRONMENT

MongoDB 7.0.0 / Express 5.1.0 / driver 6.20.0

VERSION-SPECIFIC DOCUMENTATION

MongoDB 8.0 deprecates server-side JavaScript functions

MongoDB 8.0 server-side JavaScript documentation; measured lab environment

Findings and engineering lessons

01 Client-owned operators can change credential checks

02 String validation stops the tested attack before a query

03 A fix must preserve legitimate authentication

Flexible filters need enforceable access policies

How can useful client-selected filters preserve tenant and ownership restrictions?

Scope: one local stack, synthetic records, 15 request cases

Application-owned query structure protects the intended security decision

Appendix: exact environment

Component	Recorded value
MongoDB server	7.0.0
Node.js	v23.6.1
Express / MongoDB driver	5.1.0 / 6.20.0
Transport and parser	Local HTTP / application/json / express.json
Hash-only account store	scrypt N=131072, r=8, p=1
Evidence coverage	15 cases / 5 configurations / 2 runs

Appendix: query mechanism and session decision

```
const filter = {
  username: req.body.username,
  password: req.body.password
};
const account = await users.findOne(filter);
const authenticated = Boolean(account);
if (authenticated) createSession(account);
```

Deliberately vulnerable excerpt. Synthetic plaintext account collection.

Appendix: hardened credential path

```
if (!validContract(req.body)) return reject(400);

const account = await accounts_hashed.findOne({
  username: req.body.username
});

const ok = await verifyPassword(
  req.body.password, account?.passwordHash ?? dummyHash
);
if (account && ok) createSession(account);
```

Separate hash-only records. No plaintext password in accounts_hashed.

Appendix: case results C01-C08

Case	Request shape	Toy modes	Validated / hardened
C01	Valid credentials	200	200
C02	Wrong string password	401	401
C03	Unknown username	401	401
C04	Password operator object	200	400
C05	Username operator with wrong password	401	400
C06	Operator-looking literal string	401	401
C07	Null password	401	400
C08	Array password	401	400

Appendix: case results C09-C15

Case	Request shape	Toy modes	Validated / hardened
C09	Missing password	401	400
C10	Numeric password	401	400
C11	Extra field with valid credentials	200	400
C12	Top-level array	401	400
C13	Both fields contain operators	200	400
C14	Empty password	401	400
C15	Empty username	401	400

Appendix: independent configuration checks

Control	Independent observation
Authenticated read-only account	labreader / read role on cs440_lab
Read permission	Alice record lookup succeeds
Write restriction	Insert fails: code 13 / Unauthorized
JavaScript enabled	Harmless $where predicate returns Alice
JavaScript disabled	Same $where fails: code 6108304

C04 still creates a session in both control configurations

Appendix: captured authentication bypass

C04 / baseline: Alice session; protected notes accessible

DATABASE PREDICATE

```
{
  "username": "alice",
  "password": {
    "$ne": ""
  }
}
```

APPLICATION DECISION

```
{
  "authenticated": true,
  "account": "alice"
}
```

Alice: private synthetic project notes

Appendix: captured hardened rejection

C04 / hardened: no query; no session; protected notes denied

DATABASE PREDICATE

```
Rejected before filter construction
```

Database query: none

APPLICATION DECISION

```
{
  "authenticated": false,
  "account": null
}
```

Protected notes: access denied

Appendix: limits and further investigation

Established

Operator bypass in this toy login
Control behaviour in the recorded stack
Rejection before a find command
Preservation of valid login

Outside the evidence

Deployed prevalence
Universal NoSQL behaviour
Arbitrary administrator access
Complete production security

Next study: enforce mandatory ownership predicates in flexible search APIs

References: assigned article and database semantics

Ron, A., Shulman-Peleg, A. & Puzanov, A. (2016)
Analysis and Mitigation of NoSQL Injections
IEEE Security & Privacy, 14(2), 30-39
DOI: 10.1109/MSP.2016.36

IBM Research publication record
research.ibm.com/publications/analysis-and-mitigation-of-nosql-injections

MongoDB: $ne query predicate
MongoDB 8.0: server-side JavaScript deprecation
Authors' accessible republication: InfoQ, January 2017

References: defences and project evidence

OWASP NoSQL Security Cheat Sheet
OWASP Password Storage Cheat Sheet
MongoDB Built-In Roles documentation

```
Project evidence
nosql-lab/evidence/results.json
nosql-lab/evidence/results.csv
nosql-lab/src/server.mjs
nosql-lab/scripts/experiment.mjs
```